

测试时训练在小样本学习中的惊人有效性

Ekin Akyürek¹ Mehul Damani^{*1} Adam Zweiger^{*1} Linlu Qiu¹ Han Guo¹ Jyothish Pari¹
Yoon Kim¹ Jacob Andreas¹

摘要

语言模型（Language Models, LMs）在其训练分布内的任务上表现出色，但即使给定少量上下文任务示例，也常常难以应对结构新颖的任务。本文研究了测试时训练（Test-Time Training, TTT）——在推理过程中利用由上下文示例导出的损失临时更新模型参数——作为提升语言模型推理与小样本学习能力的机制。在抽象与推理语料库（Abstraction and Reasoning Corpus, ARC）上，利用上下文示例进行测试时训练，相较于微调基线，准确率最高可提升 6 $\times$ ，在公开验证集上使用 80 亿参数的语言模型达到 53.0%，与程序综合方法集成时达到 61.9%，与人类平均表现持平。在 BIG-Bench Hard (BBH) 上，对上下文示例进行测试时训练在 10 样本设置下比标准小样本提示高出 7.3 个百分点 (50.5% 至 57.8%)。本文的研究结果凸显了上下文学习在应对新颖任务时的局限性，并展示了测试时训练在增强语言模型适应性方面的潜力。

1. 引言

大规模神经语言模型（Language Models, LMs）在预训练期间见过的相关任务的小样本学习上，以及这些任务的基本变体或组合上，已展现出显著的成功（Brown et al., 2020; Todd et al., 2024）。当给定自然语言规范或少量示例时，语言模型通常能够推断出所需任务并生成适当的输出。然而，一个悬而未决的问题是，这些模型是否能够真正习得未经训练的新技能——尤其是在与训练分布差异显著的领域中涉及非平凡推理、规划与抽象的任务。

^{*} 同等贡献 ¹ 麻省理工学院。通讯作者：Ekin Akyurek <akyurek@mit.edu>。

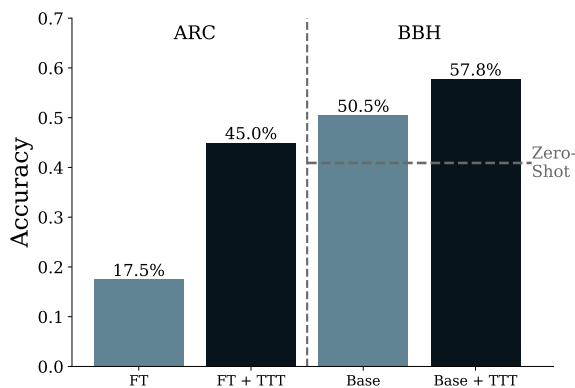


图 1. 在随机选取的 80 个 ARC 验证任务子集上的 Pass@2 准确率，以及在 BIG-Bench Hard 上的总体准确率。零样本基线在 ARC 上为 0，在 BBH 上为 40.9%，由虚线表示。TTT 将微调模型 (FT) 在 ARC 上的性能提升了 27.5 个百分点，并将 BBH 上的准确率提高了 7.3 个百分点。

偏离其预训练分布。这一问题对于理解语言模型能否以及如何展现出灵活的、新颖的技能习得至关重要，而这种能力已被提议作为智能的一种测度（Chollet, 2019）。

解决复杂且新颖的任务对语言模型而言仍极具挑战性，简单的采样方法在此类问题上往往性能不佳（Wu et al., 2024; McCoy et al., 2024）。然而，近期进展表明，通过增加额外的测试时计算，语言模型可以得到显著改进。多种方法属于此类，例如思维链提示（Wei et al., 2022）、带多数投票的采样（自洽性；Wang et al., 2023）、代码执行（Brown et al., 2024; Snell et al., 2025; Damani et al., 2025）以及搜索（Yao et al., 2023）。

在测试时使用实例特定数据更新模型参数的想法，源于局部学习（Bottou & Vapnik, 1992）和直推学习（Joachims, 1999）的文献。在这些方法中，学习器在观察到测试输入后细化其参数或假设，以适应单个示例或小簇示例。此类方法本质上模糊了训练与推理之间的界限，并能在低数据场景或分布偏移下实现稳健的自适应。

这些直推思想在深度神经网络中的现代版本被广泛称为测试时训练。在 TTT 中，模型在推理时仅使用当前测试实例或一小批测试实例进行更新，通常通过显式梯度步骤实现。虽然测试时自适应已在视觉模型（Sun et al., 2020）和序列架构（Gandelsman et al., 2022; Sun et al., 2024; Behrouz et al., 2025）中得到探索，但其与其他小样本学习技术——尤其是上下文学习——的相互作用仍未被充分理解。

本文探讨了如何在标准上下文学习（In-context Learning, ICL）之上利用测试时训练（Test-Time Training, TTT）来提升需要推理或基于规则的泛化的挑战性任务上的性能。上下文学习是一种无需参数更新的强大适应手段，由简短的任务特定提示引导。我们表明，将 ICL 与基于测试数据的显式梯度更新相结合，可以显著提升特别困难任务上的性能。具体而言，我们的主要贡献¹如下：

1. 对有效测试时训练关键组件的系统分析，包括推理时选择训练数据的策略、训练目标，以及 TTT 如何与语言模型的预训练参数和上下文学习相互作用。
2. TTT 在两个具有挑战性的基准套件上的应用——抽象与推理语料库（Abstraction and Reasoning Corpus, ARC; Chollet, 2019）和 BIG-Bench Hard（BBH; Srivastava et al., 2023; Suzgun et al., 2023）。

在 ARC 上，我们的 TTT 方法优于现有的开源神经方法，使用 8B 模型达到 53.0 % 的准确率，与程序合成方法集成时达到 61.9 %（与人类表现相当）。在 BBH 上，TTT 相比少样本提示带来 7.3 % 的绝对提升，达到 57.8 % 的准确率。在涉及结构规则或分布偏移的任务（例如 Dyck 语言、Ruin 名称）上提升尤为显著，TTT 相比标准上下文提示带来 20 – 50 个百分点的提升。

总体而言，我们的发现强调 TTT 大幅提升了语言模型在分布外任务上的少样本学习能力。

2. 预备知识

2.1. 上下文学习

在特定规模下，许多语言模型展现出无需更新参数即可适应新任务的能力，只需基于提供的输入示例或指令进行条件化。给定

¹Code and data are available at <https://github.com/ekina/kyurek/marc> (ARC) and <https://github.com/adamzweiger/Fewshot-TTT> (BBH).

一系列输入-输出对 $(x_1, y_1), \dots, (x_k, y_k)$ 和一个新输入 x_{k+1} ，语言模型可以通过从以下分布采样来生成相应的输出 $\hat{y}_{k+1}$ ：

$$\hat{y}_{k+1} \sim \text{LM}(\cdot \mid x_1, y_1, \dots, x_k, y_k, x_{k+1})$$

虽然先前工作（Akyürek 等人, 2023）讨论了上下文学习（ICL）作为隐式机器学习仿真的可能性，但经验证据表明，语言模型的上下文学习并不总是类似于标准机器学习算法（Zhao 等人, 2024; Min 等人, 2022b）。此外，上下文学习在“开箱即用”处理新任务时常常遇到困难。例如，大语言模型在 ARC 等数据集上表现不佳（Opielka 等人, 2024; Bober-Irizar 和 Banerjee, 2024）。

2.2. 测试时训练

测试时训练（TTT）使参数化模型能够在推理过程中通过针对每个测试输入动态更新参数来进行适应。在大语言模型时代，这种方法仍然相对未被探索。一般的测试时训练过程如下：从初始模型参数 θ_0 开始，对于每个测试输入（或一批输入） d ，我们生成一个临时训练数据集 $\mathcal{D}_{\text{TTT}}$ ，然后优化这些参数以最小化损失函数

$$\arg \min_{\theta} \sum_{d_{\text{TTT}} \in \mathcal{D}_{\text{TTT}}} \mathcal{L}(\text{LM}(d_{\text{TTT}}; \theta)),$$

得到临时更新的参数 θ_d ，随后用于预测。²在先前工作中（例如，Sun 等人, 2020）， $\mathcal{D}_{\text{TTT}}$ 通常通过将无监督目标（例如，掩码自编码）应用于输入 x 本身来构建。在本文中，我们将测试时训练扩展到小样本学习设置，将其视为一种直推式学习形式，利用小样本演示示例来改进预测。尽管测试时训练也可以应用于思维链（CoT; Wei 等人, 2022），但我们专注于直接直推，其中演示由输入-输出对 (x, y) 组成，不包含中间推理步骤或显式函数描述。

我们考虑的小样本学习设置以演示对

$(x_1, y_1), \dots, (x_K, y_K)$ 的形式提供了更丰富的上下文。测试时训练的一种简单方法是直接输入输出训练，其中我们直接将每个输入输出 (x_k, y_k) 对视为训练实例。我们的关键洞见是，小样本示例还可以用于构建更稳健且更广泛的合成上下文学习任务 $\mathcal{D}_{\text{TTT}}$ ，从而在测试时实现有效的模型适应。²注意，此处“测试时训练”的用法与近期一系列工作相关但不同，在这些工作中，循环神经网络的隐藏状态被视为参数，更新方程被解释为优化基于召回率的回归目标（Ravi & Larochelle, 2017; Sun et al., 2024; Behrouz et al., 2025; Wang et al., 2025）。

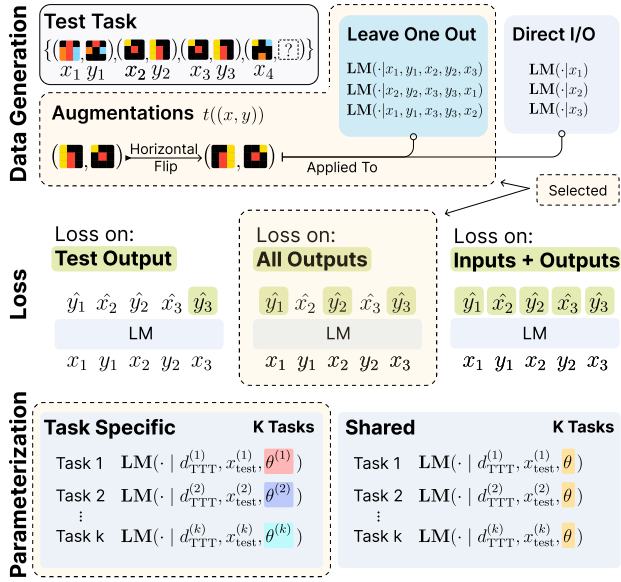


图 2. 测试时训练设计决策。数据生成：测试任务由输入输出对 $\{(x_i, y_i)\}$ 组成。留一法策略每次移除一个示例以形成上下文学习任务，而增强进一步扩展数据集。另一种直接输入输出方法直接在示例上进行训练。损失：模型使用在测试输出（仅测试时预测）、所有输出（包括演示输出）或输入和输出（所有标记）上计算的损失进行训练。参数化：任务特定方法为每个任务训练单独的适配器，而共享方法跨多个任务训练单个适配器。

测试时间。此外，当任务特定知识可用时，可以利用这种结构进一步扩展数据集，正如我们在 ARC 上的实验所展示的（第 4 节）。我们还探索了不使用任务特定信息的一般情况，并在 BBH 上进行了测试（第 5 节）。

本文的实验描述了测试时训练流程的每个组成部分，研究了以下阶段的不同设计选择：（1）在测试时构建输入特定的训练数据集 $\mathcal{D}_{\text{TTT}}$ ；（2）通过优化数据集 $\sum_{d \in \mathcal{D}_{\text{TTT}}} \mathcal{L}(\text{LM}(d; \theta))$ 上的损失函数 $\mathcal{L}$ 来微调语言模型；（3）使用基于自洽性的增强推理策略从更新后的模型中采样以获得最终预测。

3. 测试时训练设计

本节讨论将测试时训练（Test-Time Training）应用于大语言模型（Large Language Models）的关键设计选择与挑战，包括如何最佳利用其上下文学习能力、如何组织数据以实现高效处理、采用何种优化目标，以及如何高效更新模型参数。我们在测试时训练数据集的构建和优化设置中详细阐述了这些考量（图 2）。

3.1. 数据生成

Given a task with K training input-output pairs $\{(x_k, y_k)\}_{k=1}^K$, a *test-time training dataset* $\mathcal{D}_{\text{TTT}}$ can be created by either following an in-context learning setup or a direct input-output (direct I/O) setup (top row in Figure 2):

留一法任务 我们从留一法上下文学习任务开始。对于每一对 (x_j, y_j) ，我们将其从演示集中排除，并在新构建的合成任务中将其视为“测试”示例：

$$d_j^{\text{ICL}} = \left(\{(x_k, y_k)\}_{k \neq j}, x_j, y_j \right).$$

此处， $\{(x_k, y_k)\}_{k \neq j}$ 作为“上下文演示”， (x_j, y_j) 是“合成测试示例”。为增加合成任务的数量，我们进一步对每个 d_j^{ICL} 中的演示顺序进行排列。

直接输入输出任务 我们并非构建上下文任务，而是将每个 (x_k, y_k) 对独立视为单个训练实例：

$$d_j^{\text{I/O}} = (x_j, y_j).$$

在此设置下，模型在这些训练对上微调，不包含上下文演示。尽管该方法计算效率更高，但我们的结果（第 4 节和第 5 节）表明，其性能不如利用上下文演示的方法。

数据增强 对于具有结构化输入的特定任务（例如，抽象推理语料库（ARC）），我们可以应用可逆变换（例如，翻转、旋转、颜色置换）来进一步扩充测试时训练（TTT）数据集。设 $\mathcal{T}$ 为一组可逆变换。对于每个 $t \in \mathcal{T}$ ，我们有 $t^{-1}(t(x)) = x$ ，因此我们可以将 t 应用于 d_j 中的每个训练和测试实例，以生成变换后的任务 $t(d_j)$ 。由于这些变换保留了数据中的核心关系（例如，输入-输出模式相同，只是旋转了），它们有效地扩展了训练信号。如果使用基于规则的变换，最终的 TTT 数据集为： $\mathcal{D}_{\text{TTT}} = \bigcup_{t \in \mathcal{T}} \bigcup_j t(d_j)$ 。

3.2. 损失函数

我们在 $\mathcal{D}_{\text{TTT}}$ 上优化标准语言模型（LM）损失。对于上下文留一法设置，我们尝试了三种不同的损失计算方式（图 2 中间行）：

- 测试输出（无演示损失）标准形式，损失在 y_{test} 上计算：

$$\mathcal{L}_{\text{LM}}^{\text{label}} = \mathcal{L}_{\text{LM}}(y_{\text{test}} | x_1, y_1, \dots, x_K, y_K, x_{\text{test}}; \theta)$$

- 所有输出 3 除了测试输出上的损失外，3 对于 ARC，我们从 $k=2$ 开始索引，因为如果不观察至少 1 个演示，就无法推断 ARC 任务的底层变换。

损失还在上下文演示的输出上计算，这鼓励模型在看到之前的演示后正确预测演示输出：

$$\mathcal{L}_{\text{LM}}^{\text{outputs}} = \mathcal{L}_{\text{LM}}^{\text{label}} + \sum_{k=1}^K \mathcal{L}_{\text{LM}}(y_k | x_1, y_1, \dots, x_k; \theta)$$

• 输入和输出上的损失损失在所有标记上计算，鼓励模型学习 x 的结构

以及 y ：

$$\mathcal{L}_{\text{LM}}^{\text{all}} = \mathcal{L}_{\text{LM}}^{\text{outputs}} + \sum_{k=1}^K \mathcal{L}_{\text{LM}}(x_k | x_1, y_1, \dots, y_{k-1}; \theta)$$

该方法要求学习器同时生成任务输入与输出，这与现有的无监督测试时训练目标类似（Sun 等人，2020）。

我们在第 4.3 节和第 5.3 节中发现，第一种方法（对演示输出和测试输出同时计算损失）效果最佳。

3.3. 参数化

一旦我们获得了测试时训练数据集 $\mathcal{D}_{\text{TTT}}$ （通过上下文或直接输入输出方法构建），我们会对任务特定的低秩适配器（LoRA Adapters）执行少量梯度步骤（Hu 等人，2022）。这种方法在保持模型通用能力的同时，实现了计算高效的适配。默认情况下，我们在测试时为每个 ARC 或 BBH 任务学习任务特定的低秩适配器。也就是说，我们获得 K 个不同的低秩适配器，其中 K 是测试任务的数量。我们还尝试使用一个共享的低秩适配器，该适配器基于从多个任务中抽取的少样本示例聚合数据集进行训练（图 2 底部一行）——这是元上下文学习（Meta-ICL）的测试时版本（Min 等人，2022a）。我们发现共享适配器在 ARC 上降低了性能，而在 BBH 上提升了性能。我们在第 5.3 节中对此进行了更详细的讨论。

4. 抽象与推理语料库

4.1. 背景

抽象与推理语料库（Abstraction and Reasoning Corpus, ARC）旨在通过语言模型解决视觉谜题的能力来评估其抽象推理能力。每个谜题（以下称为任务）由二维网格（最大尺寸为 30×30 ）的输入输出对组成，网格中包含最多 10 种不同颜色的形状或图案，如图 3 所示。每对输出是通过应用一个直观且共享的变换或规则 $y = f(x)$ 得到的。每个任务包含 2-7 个演示示例和 1-3 个测试示例。

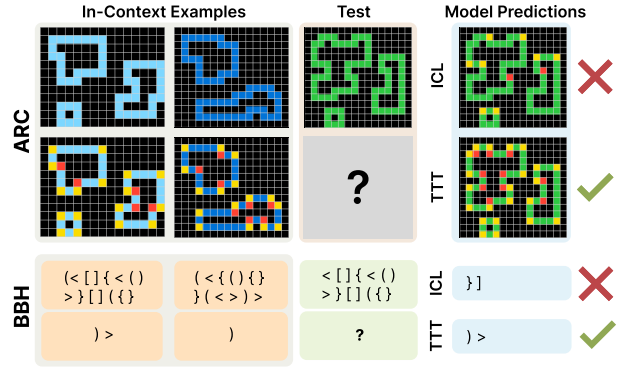


图 3. 仅在应用测试时训练后，模型成功解决的抽象与推理语料库（ARC）和大基准测试（BBH）任务示例。

4.2. 实验细节

模型架构与优化 在消融实验中，我们采用具有 10 亿参数的羊驼 3.2 模型（Llama-3.2, Llama 团队，2024 年）。在 4.6 节的最终结果中，我们使用 80 亿参数的羊驼 3 模型。我们利用低秩适配（Low-Rank Adaptation, LoRA; Hu 等人，2022 年）进行参数高效的测试时训练。更多细节见附录 C.2。

测试时训练前的微调 尽管测试时训练提供了任务特定的适应性，基础模型的初始能力显著影响其最终性能（4.4 节）。我们开发了多种生成训练数据的方法，通过微调增强基础模型的抽象推理能力，探索了自动化和半自动化的任务生成方法。这与测试时训练是互补的，因为当应用测试时训练时，基础模型是在与测试任务不同的任务上进行微调的。关于我们的数据生成策略，以及不同数据来源和模型规模对性能的影响，详见附录 B。微调后的基础模型作为所有后续实验的基础。

评估 成功标准要求对所有测试输出产生精确匹配（无部分得分）。遵循标准的抽象与推理语料库评分标准，我们使用两次尝试通过率（pass@2）度量，并为每个测试输入生成 2 次尝试。原始训练集和验证集各包含 400 个任务。然而，为了高效评估，我们从抽象与推理语料库验证集中随机选取 80 个平衡任务，包括根据（LeGris 等人，2024 年）分类的 20 个简单、20 个中等、20 个困难、20 个专家任务（任务列表见附录 A.2）。除最终结果外，我们在所有实验中使用这个抽象与推理语料库任务子集。出于效率原因，我们将 $\mathcal{D}_{\text{TTT}}$ 限制为每个任务最多 250 个示例。附录 C.2 提供了超参数的额外细节。

推理 扩展推理时计算最常用的技术之一是使用温度采样来

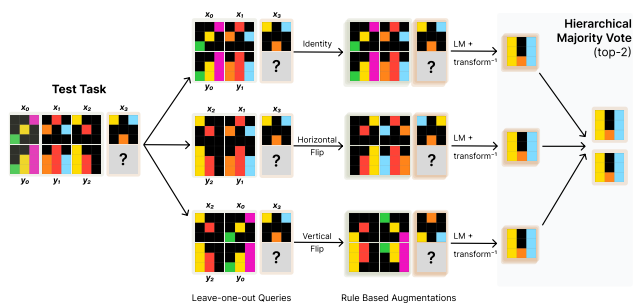


图 4. 增强推理与分层投票。我们使用留一任务和可逆几何变换来获得任务的多个等价版本以进行增强推理。来自这些版本的预测通过分层投票策略进行聚合：首先在每个变换内部进行投票，然后每个变换的顶部候选者进行全局投票，以产生前两个预测。

获得多个响应并根据排序器选择最佳响应，称为自治性（Wang et al., 2023）。然而，这在 ARC 中不可行（输出网格是直接预测的），因为无法在确保样本内部一致性的同时直接强制样本间的多样性。作为自治性的替代方法，我们尝试了一种增强推理策略，将贪婪解码与输入的多个版本相结合。具体来说，我们通过使用几何变换生成多个预测候选。然后，我们采用分层投票策略从生成的候选集中确定最终预测。该方法涉及两个投票阶段，以逐步缩小最佳候选范围：（1）

变换内投票：按相应的变换 t 对预测进行分组。在每个组内，选择出现频率最高的前 3 个预测。（2）**全局投票：**从上一步中选择的特定变换候选者中，选择所有变换中出现频率最高的前 2 个预测。增强推理流程总结于图 4，流程的完整细节见附录 E。

4.3. TTT 设计的影响

在本节中，我们将方法的最终实现与 TTT 的不同设计选择进行比较。**FT** 作为基线，仅使用带有上下文演示的微调模型。无变换省略了增强步骤。直接 I/O 数据用直接输入-输出任务公式（第 3.1 节）替换上下文任务。

共享 TTT 在所有任务中使用单个 LoRA 适配器，而非为每个任务学习一个。无演示损失移除了对演示输出的损失（第 3.1 节）。

结果如图 5 所示。我们的 TTT 方法有效，将微调模型的准确率提高了约。上下文格式尤其重要—— $6 \times (5\% \rightarrow 29\%)$

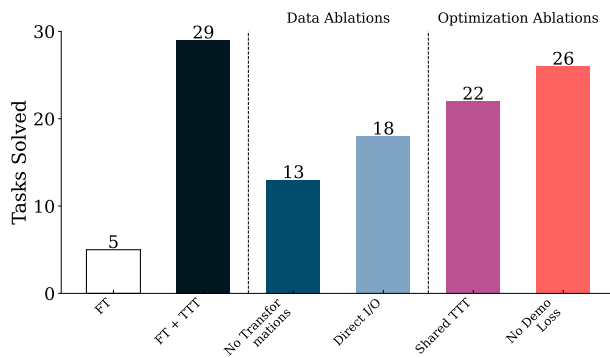


图 5. TTT 在 ARC 上不同数据与优化消融的准确率。我们的数据消融表明，ICL 数据格式对于有效的 TTT 至关重要，且对 TTT 数据集应用变换以增强数据能显著提升性能。在优化方面，学习任务特定适配器显著优于使用单个适配器，而对上下文演示计算损失则带来轻微的性能提升。

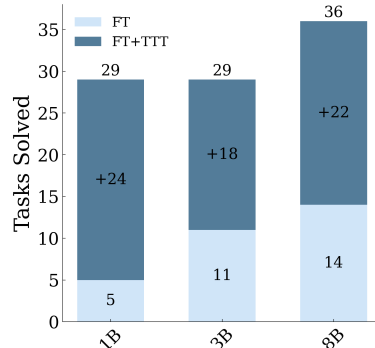


图 6. 不同模型规模下的性能结果。微调模型的性能随规模增大而提升。然而，TTT 之后的扩展行为不太明确。例如，1B 和 3B 模型的最终性能在 TTT 后完全相同。

重要：使用直接输入-输出数据构建 $\mathcal{D}_{TTT}$ 导致任务数下降 11 个（38%）。移除变换导致任务数下降 16 个（55%）。在优化方面，逐任务 LoRA 适配器比单个共享适配器多解决 7 个任务（24%）。对演示输出计算损失带来适度但一致的增益（26% $\rightarrow$ 29%）。

4.4. 模型规模的影响

我们使用合成生成的数据对 1B 和 3B 的 Llama 3.2（指令微调版）以及 8B 的 Llama 3（指令微调版）进行全量微调，详见附录 B，然后采用默认的测试时训练（TTT）实现。图 6 展示了不同模型规模下的结果。增大模型规模持续提升微调性能，其中 8B 模型达到了 36% 的最高准确率。在所有模型规模下，TTT 均带来显著的性能提升。我们还观察到，对于较小的模型规模，TTT 有效地缩小了性能差距，1B 和 3B 模型在 TTT 后达到了相近的准确率。

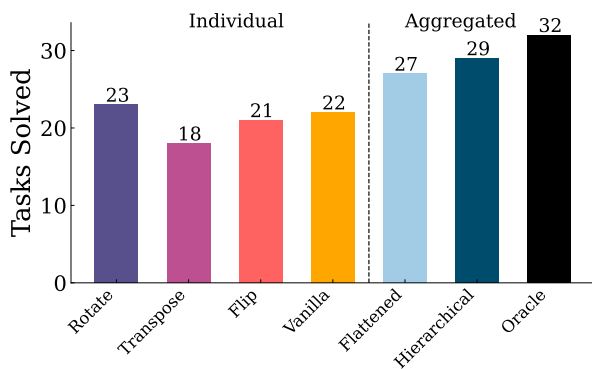


图 7. 不同变换和投票方案的准确率。虽然单个变换通常表现适中且彼此相当，但通过投票对它们进行聚合能带来显著提升。值得注意的是，具有两个投票阶段的分层投票策略优于扁平投票方法。我们的分层方法接近预言机水平，证明了其在正确答案存在时准确选择答案的有效性。

4.5. 增强推理的影响

为了分析增强推理和投票的影响，我们进行了多项消融实验：（1）基线方法，不进行变换或高级投票，直接生成两个预测；（2）

变换推理，应用单一变换（旋转、转置或翻转）以衡量其孤立效果；

（3）分层投票，我们的完整流程，结合增强推理和结构化投票；（4）扁平投票，从所有生成输出的一轮投票中选取前 2 个预测；以及（5）预言机，作为上限，在正确答案存在时直接选择它。

如图所示，单独变换的效果有限（其中转置表现最差），但它们的聚合显著提升了结果。分层投票进一步优于扁平化投票方法，并接近预言机的准确率，这表明我们的两阶段聚合能够有效识别出正确的解决方案（当它存在时）。

4.6. 与其他系统的比较

在 80 个任务上的实验之后，我们展示了在完整的 ARC 公共评估集上的综合结果，将我们的系统与现有方法进行比较。我们的分析集中在三个关键方面：我们的 TTT 方法论的影响、将我们的方法与现有方法结合的好处，以及全神经网络方法与程序合成方法之间的差异。

TTT 我们将 TTT 和增强推理过程应用于我们的基础微调模型（微调的 8B 模型）。TTT 显著地将准确率从 18.3 % 提高到 47.1 %。

与现有方法的集成 Li 等人（2025）的一项并行工作引入了 BARC，实现了 54.4 % 的准确率

PS	Fine-tuned LM	TTT Method	Score
X	Ours	X	18.3%
X	Ours	Ours	47.1%
X	BARC	Ours	53.0%
BARC	Ours	Ours	58.5%
BARC	BARC	Ours	62.8%
Avg. Human			60.2%
Best Human			97.8%
BARC (ensemble)			54.4%
BARC (no synthesizer)			39.3%
Claude 3.5 Sonnet			21.0%
GPT-4o			9.0%
OpenAI o1 preview			21.0%
DeepSeek r1			20.5%
OpenAI o3			82.8%

表 1. 不同系统在 ARC 验证集上的 Pass@2 得分。我们的 TTT 流程持续改进了基础模型。当应用于我们的微调模型时，我们实现了 47.1% 的准确率；当应用于 BARC 模型（Li 等人，2025）时，实现了 53.0% 的准确率。我们将我们的方法与基于程序合成（PS）的模型进行集成，达到了 61.9 % 的得分，与人类平均表现 60.2% 相当。

通过结合神经和程序合成方法实现了准确率。虽然他们的全神经方法与我们的系统有相似之处，但我们的 TTT 和推理流程具有几个额外的组件（逐任务 LoRA、更多增强、分层投票），这些组件提升了性能。为了验证我们的改进，我们将我们的 TTT 流程应用于 BARC 的全神经模型，实现了 53.0% 的准确率——比他们原始的 TTT 方法提高了 35%。

基于这些结果，我们探索了本文方法与 BARC 的组合。将本文的 TTT 流程和神经模型与 BARC 的合成器结合，准确率提升至 58.5 %。将本文的 TTT 流程与 BARC 的神经模型和合成器结合，准确率提升至 61.9 %。该配置在基准上达到了 60.2%（LeGris 等人，2024）的平均人类性能。

比较程序生成与端到端建模时，Li 等人（2025）发现，针对 ARC 的程序合成与全神经预测器高度互补。他们的端到端神经模型仅能解决程序合成模型所解决任务的 42.2%。然而，我们发现，当配备我们的 TTT 流水线后，BARC 微调的全神经模型解决了程序合成模型所解决任务的 73.5%。这表明我们的 TTT 流水线显著提升了神经模型学习系统化推理模式的能力，使其更接近程序合成模型所捕捉的模式。

半私有评估 ARC-AGI 挑战提供了一个隐藏的“半私有数据集”，并对提交的方案进行外部测试。我们将集成解决方案提交至官方 ARC-AGI 半私有评估，观察到 47.5 % 的准确率。这一下降可能归因于更多

在半私有评估数据集中存在显著的分布偏移。若半私有评估结果未来得以公开，我们将对这些性能差异进行详细分析。

5. 大基准难题 (BIG-Bench Hard)

5.1. 背景

BIG-Bench Hard (BBH; Srivastava 等人, 2023; Suzgun 等人, 2023) 是一个基准, 包含 23 种任务类型中的 27 个挑战性任务, 旨在评估大语言模型在推理、组合性和泛化方面的能力。与 ARC 不同, BBH 具有更广泛的自然语言结构, 且缺乏统一的输入格式, 因此不适合可逆变换。然而, 这一更广泛的范围为在更一般化的环境中评估 TTT 的有效性提供了有价值的测试平台。尽管缺少可逆变换——此前在 ARC 中用于扩展 TTT 数据集并增强推理——TTT 仍显著提升了在 BBH 上的性能。

5.2. 实验细节

模型架构与优化 我们使用 Llama 3.1 (8B; Llama 团队, 2024)。对于每个任务 d , 我们在测试时训练一组独立的 LoRA 参数, LoRA 秩为 64, 对演示对进行 40 次随机打乱以生成留一上下文任务。更多超参数细节见附录 F.1。

在 BIG-Bench Hard 上, 我们的基础语言模型开箱即能取得非平凡分数。因此, 与 ARC 不同, 我们不在 BBH 之外的合成任务上进行任何初始微调。此外, 由于模型在零样本设置下已能取得非零性能, 我们提供零样本结果, 并分析 TTT 和 ICL 如何在其基础上改进。

评估 对于 BBH 中的 27 个任务, 我们考虑 10 样本设置, 即从每个任务的数据集中随机选择 10 对作为演示对, 并在剩余数据上进行评估。这 27 个任务中的每一个都类似于单个 ARC 任务, 由测试时给定的 10 个带标签示例作为演示对组成。我们报告五个随机种子的平均结果, 每个种子指定哪 10 个示例构成演示子集。为了更好地控制测试时训练的评估过程, 我们编写了自己的评估函数, 该函数可在我们的代码库中找到 (更多细节见附录 F.1)。每个任务的评估示例数量为 240, 除了三个任务: 因果判断、表格中的企鹅和讽刺, 它们分别有 177、136 和 168 个评估示例。请注意, 与 ARC 相比, 每个任务的评估样本数量较多, 这意味着我们可以进行任务特定分析, 以分析哪些类型的任务

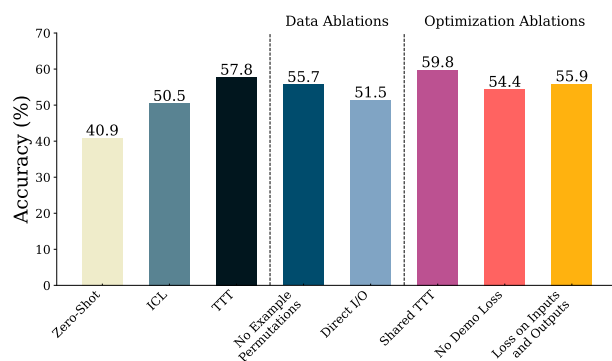


图 8. BIG-Bench Hard 总体结果。TTT 比标准上下文学习高出 7.3 个百分点, 从 50.5% 提升到 57.8%。我们相对于直接输入-输出数据的性能提升表明, 使用上下文留一任务至关重要。不采用演示损失或在输入上计算损失会导致性能下降。与 ARC 不同, 在所有任务上使用共享适配器可提升性能。

从 TTT 中获益最多 (第 5.4 节)。与 ARC 不同, 我们没有可逆变换集合来运行增强推理。相反, 我们使用贪心解码。更多超参数细节和评估细节见附录 F.2。

5.3. TTT 设计的影响

在本节中, 我们评估我们的方法及其消融, 主要比较零样本基线、ICL 和 TTT。

无示例置换 在单个上下文提示上更新模型, 而非多个打乱版本。直接输入输出将每个输入输出对视为独立训练实例。共享测试时训练使用跨任务的单一适配器, 而非任务特定适配器。无演示损失移除应用于演示输出的损失。损失

作用于输入与输出 将损失计算扩展至输入与输出。这些消融实验详见第 3 节。由于结果为 5 次运行的平均值, 各方法均值的标准误差见附录 F.1, 平均 0.4%。

图 8 结果显示, 测试时训练达到 57.8% 的总体准确率, 优于标准上下文学习 (50.5%) 和直接输入输出学习 (51.5%)。这表明测试时训练的能力不仅限于抽象推理语料库, 还扩展到更多样、更复杂的推理任务, 证明其在更广泛的自然语言问题求解场景中的有效性。

我们观察到, 无示例置换的测试时训练——在推理前对单个上下文提示执行多次梯度步骤——将准确率降至仍令人印象深刻的 55.7%。仅在测试输出上计算损失将准确率降至 54.4%, 而将其应用于输入与输出则达到 55.9%。

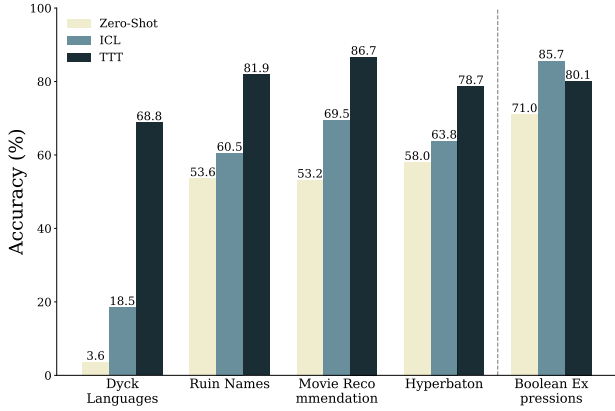


图 9. 测试时训练与上下文学习得分差异最大的任务的 BIG-Bench Hard 结果。左侧四个任务显示测试时训练相比上下文学习最显著的改进，右侧任务相对上下文学习的测试时训练得分最低。完整的任务特定结果见附录 F.2。

共享适配器 与在抽象与推理语料库 (ARC) 上不同，使用共享适配器提升了在大型基准测试 (BBH) 上的性能，表明 BBH 中的任务在训练期间不会相互干扰。在 ARC 数据集上，每个谜题具有相同的输入格式，因此区分多个任务较为困难，并且使用单一适配器可能会产生冲突的梯度。然而在 BBH 中，区分任务十分简单（指令以纯文本形式不同），且许多任务相互促进。例如，在“五物体逻辑推理”上更新也有助于“三物体逻辑推理”，而不会损害“单词排序”。尽管这不再是测试时对每个呈现的不同任务进行训练，但可以解释为在测试时对整个数据集进行集体呈现的测试时训练 (TTT)。

5.4. 任务特定分析

我们的任务特定结果表明，TTT 带来的性能提升高度依赖于任务。在 BBH 的 27 个任务中，与上下文学习 (ICL) 相比，TTT 仅在 2 个任务上导致性能下降至少 2%。相比之下，12 个任务显示出至少 2% 的提升，其中 9 个任务提升至少 5%。图 9 展示了 TTT 相较于 ICL 或零样本学习性能提升最显著四个任务以及性能下降最显著的一个任务。按 TTT 相对于 ICL 的提升幅度排序，这些任务依次为：戴克语言（括号匹配）、名字趣味化（幽默名称修改）、电影推荐（选择相似电影）、倒装（形容词排序）和布尔表达式（求值布尔表达式）。每个任务的详细结果见附录 F.2。

我们假设 TTT 带来的提升可能由涉及分布偏移和结构化模式的任务驱动。例如，像戴克语言和倒装这样的任务遵循清晰的语法或程序化规则，这可能

与 TTT 在测试时适应潜在结构规律的能力高度契合。

相反，需要显式逐步计算的任务在 TTT 下收益有限。例如，布尔表达式在 TTT 下从 85.7% 下降到 80.4%。该任务的算法性质——依赖于顺序推理而非基于模式的转换——以及其可能的预训练暴露表明，TTT 的更新可能无法解决其特定需求。尽管这些具体观察与我们的假设一致，但某些任务从 TTT 中获益更多的原因仍是一个开放问题。

6. 相关工作

测试时训练 利用实例特定数据在测试时更新模型参数的想法可追溯至早期关于局部学习的研究 (Bottou & Vapnik, 1992)。近期，Sun 等人 (2020) 提出了一种简单的测试时自监督方案，用于在面临分布偏移时调整图像分类器。在语言建模中，Hardt 与 Sun (2024) 在测试时对检索到的邻居进行微调以获得显著提升，而 Hubotter 等人 (2025) 则通过主动数据选择来优化检索。

ARC 挑战 抽象与推理语料库 (Abstraction and Reasoning Corpus, ARC; Chollet, 2019; Chollet 等人, 2025) 是一组极具挑战性的小样本视觉推理问题集合。大多数 ARC 方法主要分为两类：程序合成与全神经方法。程序合成方法 (Butt 等人, 2024; Wang 等人, 2024; Li 等人, 2025; Greenblatt, 2024) 首先尝试找到变换函数 f ，然后将其应用于测试示例。全神经方法 (Veldkamp 等人, 2023; Bober-Irizar 与 Banerjee, 2024) 尝试直接预测输出 y^{test} ，仅隐式地建模 f 。在本工作中，我们采用全神经方法，使用语言模型预测测试输出。近期研究探索了混合方法，利用推理扩展和深度学习引导的程序合成 (Greenblatt, 2024; Li 等人, 2025)。类似地，我们发现将神经模型与程序合成相结合能够提升性能。

7. 结论

我们对测试时训练进行了研究，并证明其能够显著提升语言模型在抽象推理和小样本学习任务上的性能，具体包括抽象与推理语料库 (ARC) 和 BIG-Bench Hard (BBH)。我们的主要贡献包括：一个稳健的测试时训练框架，该框架采用留一法上下文任务构建、优化设置以及测试时训练后的推理策略。我们的结果揭示了测试时训练在解决新颖推理任务方面的潜力，表明测试时方法在推动下一代语言模型发展方面具有显著前景。

局限性

优化偏差 在 ARC 的开发过程中，我们使用了一组 80 个任务进行验证/消融实验。标准超参数（学习率、训练轮次）基于该集合进行优化，这可能引入了一些偏差。

数据泄漏 虽然基础 Llama-3 在 ARC 的公开验证集上表现不佳，但该数据集的公开可用性带来了这些模型可能在预训练期间见过这些示例的可能性。同样，虽然基础模型在 BBH 上取得了合理的性能，但其公开可用性也引发了类似的担忧。

致谢

我们衷心感谢 BARC 团队（Li 等人，2025）的支持与合作，将我们的方法与他们集成，形成了对 ARC 公开集的官方联合提交。我们感谢 Aniruddha Nrusimha 在参数高效训练方面的有益讨论。这项工作得到了 MIT – IBM Watson AI Lab 以及国家自然科学基金会（资助号 IIS-2212310、IIS-2238240 和 CCF-2217064）的支持。这项工作还受益于西蒙斯研究所语言模型与变换器项目期间的多次交流。

参考文献

- Acquaviva, S., Pu, Y., Kryven, M., Sechopoulos, T., Wong, C., Ecanow, G. E., Nye, M. I., Tessler, M. H., and Tenenbaum, J. Communicating natural programs to humans and machines. In *Advances in Neural Information Processing Systems 35*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/182aed0379591ebd1d655b2bdc152075-Abstract-Datasets_and_Benchmarks.html.
- Akyürek, E., Schuurmans, D., Andreas, J., Ma, T., and Zhou, D. What learning algorithm is in-context learning? Investigations with linear models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/pdf?id=0g0X4H8yN4I>.
- Behrouz, A., Zhong, P., and Mirrokni, V. Titans: Learning to memorize at test time, 2025. URL <https://arxiv.org/abs/2501.00663>.
- Bober-Irizar, M. and Banerjee, S. Neural networks for abstraction and reasoning. *Scientific Reports*, 2024. ISSN 2045-2322. doi: 10.1038/s41598-024-73582-7. URL <https://doi.org/10.1038/s41598-024-73582-7>.
- Bottou, L. and Vapnik, V. Local learning algorithms. *Neural Computation*, 1992. ISSN 0899-7667. doi: 10.1162/neco.1992.4.6.888. URL <https://doi.org/10.1162/neco.1992.4.6.888>.
- Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., and Mirhoseini, A. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- Butt, N., Manczak, B., Wiggers, A., Rainone, C., Zhang, D. W., Defferrard, M., and Cohen, T. Codelft: Self-improving language models with prioritized hindsight replay. In *Proceedings of the 41st International Conference on Machine Learning*. PMLR, 2024. URL <https://dl.acm.org/doi/10.5555/3692070.3692267>.
- Chollet, F. On the measure of intelligence, 2019. URL <https://arxiv.org/abs/1911.01547>.
- Chollet, F., Knoop, M., Kamradt, G., and Landers, B. ARC Prize 2024: Technical report, 2025. URL <https://arxiv.org/abs/2412.04604>.
- Damani, M., Shenfeld, I., Peng, A., Bobu, A., and Andreas, J. Learning how hard to think: Input-adaptive allocation of LM computation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=6qUUGw9bAZ>.
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html.
- Gandelsman, Y., Sun, Y., Chen, X., and Efros, A. A. Test-time training with masked autoencoders. In *Advances in Neural Information Processing Systems 35*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/bcdec1c2d60f94a93b6e36f937aa0530-Abstract-Conference.html.
- Greenblatt, R. Getting 50% (SoTA) on ARC-AGI with GPT-4o, 2024. URL <https://redwoodresearch.substack.com/p/getting-50-sota-on-arc-agi-with-gpt>. Accessed 09-11-2024.

- Hardt, M. and Sun, Y. Test-time training on nearest neighbors for large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=CNL2bku4ra>.
- Hodel, M. Addressing the Abstraction and Reasoning Corpus via procedural example generation, 2024. URL <https://arxiv.org/abs/2404.07353>.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Hübotter, J., Bongni, S., Hakimi, I., and Krause, A. Efficiently learning at test-time: Active fine-tuning of LLMs. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=NS1G1UhnY3>.
- Hurst, A., Lerer, A., Goucher, A. P., Perelman, A., Ramesh, A., Clark, A., Ostrow, A., Welihinda, A., Hayes, A., Radford, A., et al. GPT-4o system card. *ArXiv preprint*, 2024. URL <https://arxiv.org/abs/2410.21276>.
- Joachims, T. Transductive inference for text classification using support vector machines. In *Proceedings of the 16th International Conference on Machine Learning*. Morgan Kaufmann Publishers Inc., 1999. ISBN 1558606122.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23. Association for Computing Machinery, 2023. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- LeGris, S., Vong, W. K., Lake, B. M., and Gureckis, T. M. H-ARC: A robust estimate of human performance on the Abstraction and Reasoning Corpus benchmark. *ArXiv preprint*, 2024. URL <https://arxiv.org/abs/2409.01374>.
- Li, W.-D., Hu, K., Larsen, C., Wu, Y., Alford, S., Woo, C., Dunn, S. M., Tang, H., Zheng, W.-L., Pu, Y., and Ellis, K. Combining induction and transduction for abstract reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=UmdotAAVDe>.
- Llama Team. The Llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Loshchilov, I. and Hutter, F. Fixing weight decay regularization in Adam, 2018. URL <https://openreview.net/forum?id=rk6qdGgCZ>.
- McCoy, R. T., Yao, S., Friedman, D., Hardy, M. D., and Griffiths, T. L. Embers of autoregression show how large language models are shaped by the problem they are trained to solve. *Proceedings of the National Academy of Sciences*, 2024. doi: 10.1073/pnas.2322420121. URL <https://www.pnas.org/doi/abs/10.1073/pnas.2322420121>.
- Min, S., Lewis, M., Zettlemoyer, L., and Hajishirzi, H. MetaICL: Learning to learn in context. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2022a. doi: 10.18653/v1/2022.naacl-main.201. URL <https://aclanthology.org/2022.naacl-main.201>.
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., and Zettlemoyer, L. Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2022b. doi: 10.18653/v1/2022.emnlp-main.759. URL <https://aclanthology.org/2022.emnlp-main.759>.
- OpenAI. GPT-4 technical report, 2024. URL <https://arxiv.org/abs/2303.08774>.
- Opiełka, G., Rosenbusch, H., Vijverberg, V., and Stevenson, C. E. Do large language models solve ARC visual analogies like people do?, 2024. URL <https://arxiv.org/abs/2403.09734>.
- Ravi, S. and Larochelle, H. Optimization as a model for few-shot learning. In *The Fifth International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=rJY0-Kc1l>.
- Snell, C. V., Lee, J., Xu, K., and Kumar, A. Scaling test-time compute optimally can be more effective than scaling LLM parameters. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=4FWAwZtd2n>.
- Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. M., Abid, A., Fisch, A., Brown, A. R., Santoro, A., Gupta, A., Garriga-Alonso, A., et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=uyTL5Bvosj>.

- Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A. A., and Hardt, M. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 2020. URL <http://proceedings.mlr.press/v119/sun20b.html>.
- Sun, Y., Li, X., Dalal, K., Xu, J., Vikram, A., Zhang, G., Dubois, Y., Chen, X., Wang, X., Koyejo, S., Hashimoto, T., and Guestrin, C. Learning to (learn at test time): RNNs with expressive hidden states, 2024. URL <https://arxiv.org/abs/2407.04620>.
- Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q., Chi, E., Zhou, D., and Wei, J. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *Findings of the 2023 Conference of the Association for Computational Linguistics*. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-acl.824. URL <https://aclanthology.org/2023.findings-acl.824>.
- Todd, E., Li, M., Sharma, A. S., Mueller, A., Wallace, B. C., and Bau, D. Function vectors in large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=AwxytyMwaG>.
- torch tune Maintainers and Contributors. torchtune: PyTorch’s finetuning library, 2024. URL <https://github.com/pytorch/torch tune>.
- Veldkamp, K., Rosenbusch, H., Thoms, L., and Stevenson, C. Solving ARC visual analogies with neural embeddings and vector arithmetic: A generalized method. *OSF*, 2023. doi: 10.17605/OSF.IO/AKP86. URL <https://osf.io/akp86/>.
- Wang, K. A., Shi, J., and Fox, E. B. Test-time regression: a unifying framework for designing sequence models with associative memory. *ArXiv preprint*, 2025. URL <https://arxiv.org/abs/2501.12352>.
- Wang, R., Zelikman, E., Poesia, G., Pu, Y., Haber, N., and Goodman, N. Hypothesis Search: Inductive reasoning with language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=G7UtIGQmj m>.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.
- Wu, Z., Qiu, L., Ross, A., Akyürek, E., Chen, B., Wang, B., Kim, N., Andreas, J., and Kim, Y. Reasoning or reciting? Exploring the capabilities and limitations of language models through counterfactual tasks. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2024. URL <https://aclanthology.org/2024.naacl-long.102>.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of Thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems 36*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract-Conference.html.
- Zhao, S., Nguyen, T., and Grover, A. Probing the decision boundaries of in-context learning in large language models. In *ICML 2024 Workshop on In-Context Learning*, 2024. URL <https://openreview.net/forum?id=r fCtCcPuSt>.

A. ARC 数据集

我们展示了开发集中的任务、数据格式以及 ARC 数据集的评估细节（可在该 <https> 链接获取）。

A.1. 数据格式

所有实验均采用数值计算库（numpy）的数组打印格式，如图 10 所示。

A.2. 用于开发的 80 项任务列表

我们使用以下（表 2）任务作为开发阶段的验证任务。

表 2. 选定的开发任务及其基于（LeGris 等人，2024）的难度等级。

ID	Level	ID	Level	ID	Level	ID	Level
0a1d4ef5	easy	762cd429	medium	e5c44e8f	hard	e99362f0	expert
692cd3b6	easy	e7639916	medium	604001fa	hard	1acc24af	expert
1da012fc	easy	e1d2900e	medium	4364c1c4	hard	f9a67cb5	expert
66e6c45b	easy	ae291af	medium	506d28a5	hard	ad7e01d0	expert
3194b014	easy	e95e3d8e	medium	2037f2c7	hard	ea9794b1	expert
963f59bc	easy	e0fb7511	medium	d5c634a2	hard	58e15b12	expert
d37a1ef5	easy	ae58858e	medium	ac605cbb	hard	891232d6	expert
358ba94e	easy	93c31fbe	medium	27f8ce4f	hard	5833af48	expert
f3cdc58f	easy	27a77e38	medium	66f2d22f	hard	4ff4c9da	expert
55059096	easy	9bebae7a	medium	3ed85e70	hard	5b692c0f	expert
c7d4e6ad	easy	9ddd00f0	medium	8b28cd80	hard	e2092e0c	expert
4b6b68e5	easy	fe9372f3	medium	d19f7514	hard	47996f11	expert
00576224	easy	69889d6e	medium	dc2aa30b	hard	34b99a2b	expert
a04b2602	easy	15663ba9	medium	f5c89df1	hard	1c56ad9f	expert
e9c9d9a1	easy	17b80ad2	medium	50f325b5	hard	e6de6e8f	expert
ef26cbf6	easy	16b78196	medium	08573cc6	hard	fea12743	expert
7ee1c6ea	easy	5b6cbef5	medium	3d31c5b3	hard	31d5ba1a	expert
e9ac8c9e	easy	40f6cd08	medium	94133066	hard	79fb03f4	expert
1a2e2828	easy	505fff84	medium	136b0064	hard	8719f442	expert
770cc55f	easy	d017b73f	medium	90347967	hard	a8610ef7	expert

A.3. 评估

我们遵循竞赛规则：如果系统两次预测（pass@2）中任意一次正确，则判定该测试正确。在报告的任务级准确率中，除非所有测试均被解决，否则不给予部分分数，但第 4.6 节的最终表格除外。

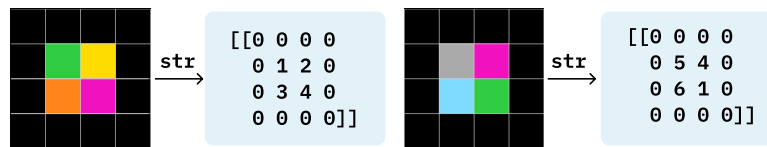


图 10. 数据格式：我们将网格转换为字符串，方法是将其表示为 0 到 10 的数字数组（numpy 数组），其中每个数字对应一种不同的颜色。

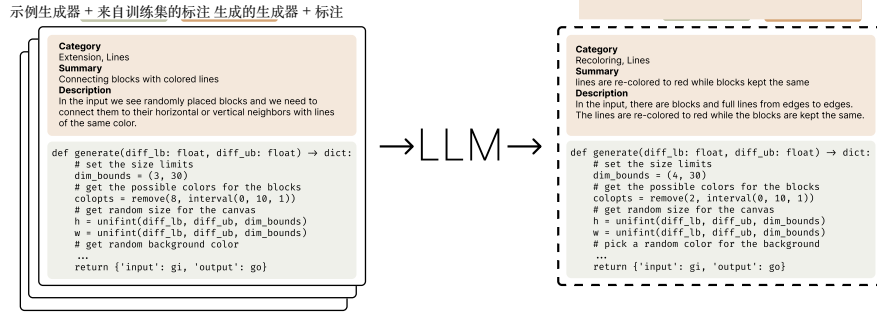


图 11. 基于大语言模型的合成任务生成：给定一些种子任务描述和 Python 任务生成器函数，我们生成更多生成器函数以产生新颖任务。我们采用三种不同方法：（1）仅使用生成器进行少样本提示，（2）使用生成器和任务描述进行少样本提示，（3）两阶段方法：首先生成自由形式描述，然后以这些描述为条件生成更多生成器（如图 12 所示）。

B. 测试时训练前的微调

虽然测试时训练有助于任务特定适应，但基础模型的能力影响最终性能。我们开发了几种生成合成训练数据的方法，通过微调增强基础模型的抽象推理能力，探索了自动化和半自动化的任务生成方法。本节详细阐述微调数据生成策略，并分析不同数据来源和模型规模对最终性能的影响。

B.1. 准备微调数据

(Hodel, 2024) 提供了领域特定语言 (DSL) REARC，以及解决任务 i 的变换 f_i 和在 $\mathcal{D}_{\text{ARC}}^{\text{train}}$ 数据集中每个训练任务用该 DSL 实现的数据生成函数 g_i 。这些函数能够采样新的输入-输出对，保持相同的底层变换原理：

$$d = (x, y) \sim \text{eval}(g_i) \quad (1)$$

其中 d 表示新生成的输入-输出对，可以使用与原始任务 i^4 相同的变换函数 f_i 求解。

(a) 使用现有生成器 生成器函数 g in RE ARC 已经通过产生同一任务的不同实例提供了有效的数据增强工具。我们通过多次运行代码并随机将这些新示例 ($d \sim \text{eval}(g_i)$) 划分为训练集和测试集，从这些训练任务中生成额外样本。这些增强示例已随其 DSL 版本提供。

(b) 少样本提示大语言模型 此外，我们采用了多种方法，利用语言模型（本文中为 GPT4 与 GPT4-o 的集成）生成新颖任务。

最简单的方法通过少样本示例生成新的任务生成器：

$$g' \sim \text{LM}(g_1, g_2, \dots, g_m) \quad (2)$$

其中 g' 是一个新的生成器函数， $g_1, \dots, g_m$ 是现有的生成器函数（如图 11 所示）。我们从现有训练集中均匀采样不同的 m 示例。重复此过程多次以获得足够数量的任务。

我们为生成器函数补充任务描述，并联合生成描述与生成器：

$$(s', g') \sim \text{LM}(s_1, g_1, s_2, g_2, \dots, s_m, g_m) \quad (3)$$

其中 s_i 表示任务 i 的描述。

⁴ 我们可以通过断言 $f_i(x) = y$ 来验证生成的示例。

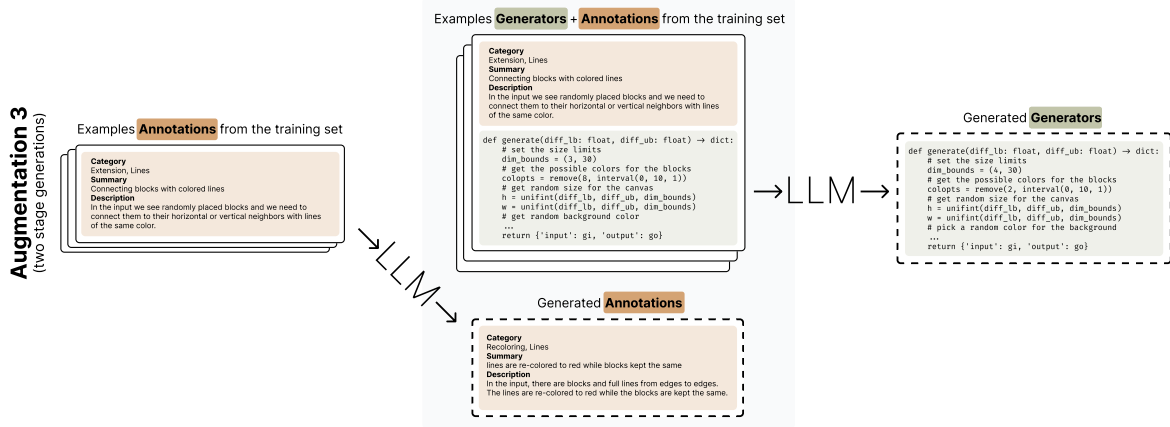


图 12. 使用大语言模型的两阶段生成：首先，我们通过少样本提示让大语言模型生成任务描述。然后，基于现有任务对和新建的描述生成新的生成器。

为获取任务描述，我们为 10 个训练任务手动创建了种子描述。随后，通过少样本提示（few-shot prompting）利用这些种子描述生成训练和验证任务描述。为增加任务多样性，我们采用具有层级字段（类别、摘要和描述）的任务描述。获取这些描述的过程见附录 D.1。

除联合生成任务描述与函数生成外，我们还部署了一种两阶段方法（图 12），描述如下：

$$s' \sim \text{LM}(s_1, s_2, \dots, s_m) \quad (4)$$

$$g' \sim \text{LM}(s_1, g_1, s_2, g_2, \dots, s_m, g_m, s') \quad (5)$$

该方法首先生成任务描述 s' ，然后基于现有任务对和新描述来条件化生成器的创建。我们通过这些基于大语言模型（LLM）的方法共收集了 6426 个生成器。图 16 展示了这些大语言模型生成任务的定性样本。

(c) 几何变换 最后，我们的合成任务通过多种几何变换得到增强，例如基本变换（旋转、反射、随机平移和尺寸缩放）、模式操作（随机补丁、平铺和重复）、颜色置换，以及涉及多个基本变换顺序应用的复合变换。这些变换以三种方式应用：

- Input grids only: $(x, y) \rightarrow (t(x), y)$
- Output grids only: $(x, y) \rightarrow (x, t(y))$
- Both input and output: $(x, y) \rightarrow (t(x), t(y))$

我们使用附录 C.1 中给出的所有变换，以及表 3 中给出的一些额外变换。在微调情况下，与 TTT 不同，我们仅对输入、仅对输出或同时对两者应用增强。这些变换以 30% 的概率随机应用于任务变体。

表 3. 我们提供了用于微调数据生成的额外增强及其函数签名和描述。

增强名称	描述
Repeat(direction, n)	将网格沿水平或垂直方向旋转 n 次。
DropoutOutput	随机删除输出网格中的一些补丁。
DropoutInput	随机删除输入网格中的一些补丁。

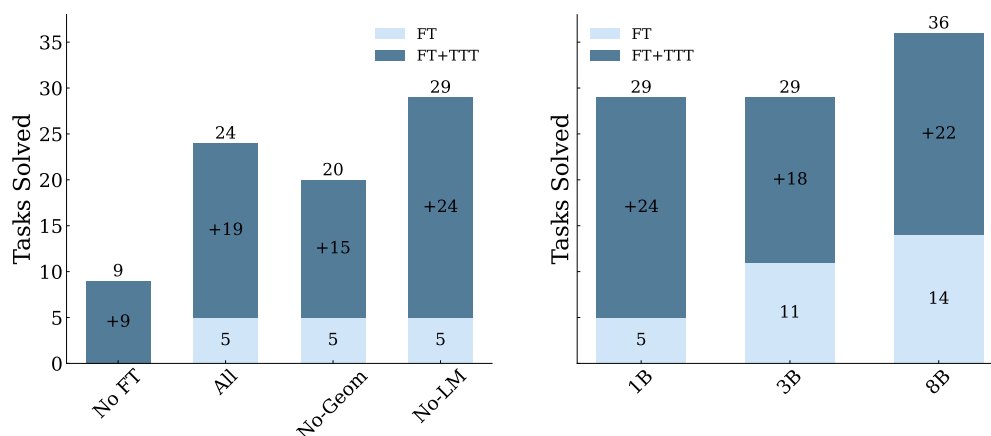


图 13. 左：使用不同数据源进行微调时的准确率。虽然所有微调模型的性能相似，但它们在测试时训练（TTT）后的性能表现出相当大的方差。正如预期，从微调数据中移除几何变换会降低性能，与使用完整数据集训练的模型相比。令人惊讶的是，从微调中排除语言模型生成的数据实际上优于使用所有数据训练的模型。右：不同模型规模下的性能结果。正如预期，基础微调模型的性能随着模型规模的增大而提高，与当前的缩放定律趋势一致。然而，TTT 后的缩放行为不太明确。例如，1B 和 3B 模型在 TTT 后的最终性能相同。完整讨论见 B.3 节。

B.2. ARC 初始微调超参数

我们使用 `torch tune` 库对 Llama-3 系列模型进行全量微调。每个模型训练最多 16000 步。1B 模型使用 2 块 NVIDIA A100 GPU，3B 和 8B 模型使用 4 块 NVIDIA A100 GPU。超参数见表 4。

表 4. ARC 初始微调超参数 超参数搜索空间

Hyperparameter	Search Space
learning rate	2.5e-5
epochs	2
batch size	32
optimizer	AdamW (Loshchilov & Hutter, 2018)
scheduler	Cosine LR Schedule with 2k warmup

B.3. 结果

我们使用增强数据对 1B、3B Llama 3.2 指令微调模型和 8B Llama 3 指令微调模型进行全量微调。格式和训练目标与第 3 节中描述的 TTT 相同。超参数详情见附录 C.2。我们对增强数据进行了以下消融实验：

1. 无微调：原始 Llama 3 指令微调模型，未进行任何微调。
2. 全部：我们使用 B.1 节中描述的所有方法，包括 REARC、基于规则的增强和语言模型生成。
3. 无几何变换：我们从所有任务中移除几何变换。
4. 无语言模型：我们仅使用 REARC 和基于规则的增强，排除由语言模型生成的任务。

我们在图 13 中展示了使用不同模型规模的结果。增加模型规模持续提升微调性能，其中 80 亿参数模型达到了 36% 的最高准确率。我们还观察到，测试时训练有效缩小了较小模型的性能差距，10 亿和 30 亿参数模型在测试时训练后达到了相似的准确率。

C. 测试时训练变换用于抽象推理语料库

我们介绍测试时训练中使用的变换及训练细节。

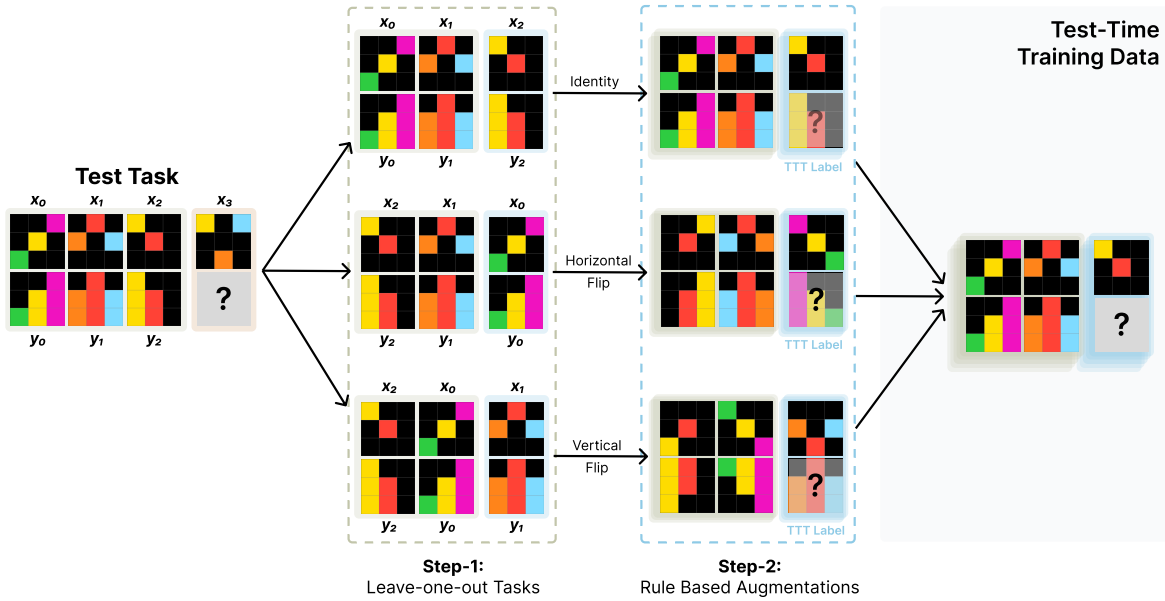


图 14. 针对测试任务的测试时训练数据集生成（第 3.1 节）：我们首先从任务的给定训练示例中创建留一任务。然后通过基于规则的变换对这些任务进行增强，以获得完整的测试时训练数据集。最后，我们在基础微调模型之上训练任务特定的低秩适配器。

C.1. 变换

我们在附录 C.1 中提供了测试时训练中使用的增强方法，具体实现请参考我们的代码库。应用这些增强后，我们还会打乱颜色和训练示例。注意，这些变换应用于所有输入和输出网格。测试时训练数据集的生成过程如图 14 所示。

C.2. 训练设置与超参数

我们使用 `torch tune` (`torch tune` 维护者与贡献者, 2024) 库在 Llama-3 系列模型上训练 LoRA 适配器。我们将 LoRA 训练应用于自注意力层的查询和值投影权重、MLP 权重以及输出投影层（在 `torch tune` 中仅适用于 Llama-3 8B）。我们在表 6 中展示了该训练的超参数。我们还发现，使用量化 LoRA 适配器 (Dettmers 等人, 2023) 代替标准（全精度）LoRA 只会导致性能小幅下降（29 → 26 个任务由 1B 参数模型解决），使其成为内存受限环境下的可行选择。

我们采用 `vLLM` (Kwon 等人, 2023) 库进行预测，因为它为我们的模型和 LoRA 推理提供了快速核函数和批处理推理。我们仅使用贪婪解码，因为在早期实验中未观察到温度采样带来的改进。我们使用 90 度、180 度旋转，水平、垂直和对角线（转置）翻转作为可逆变换。

因此，在使用 NVIDIA A100 GPU 时，对于 100 个随机采样的验证任务，整个 TTT 和推理过程大约需要 12 小时。

表 5. 我们提供了 TTT 过程中使用的增强方法及其函数签名和描述。增强名称 描述

Augmentation Name	Description
Rotate(90)	Rotates a grid 90 degrees.
Rotate(270)	Rotates a grid -90 degrees.
Rotate(180)	Rotates a grid 180 degrees.
Flip(0)	Flips a grid horizontally
Flip(1)	Flips a grid vertically
Reflect(0, reverse=True)	Flips a grid horizontally and prepends to the left of the original grid
Reflect(1, reverse=True)	Flips a grid vertically and prepends to above the original grid
Reflect(0, reverse=False)	Flips a grid horizontally and appends to the right of the original grid
Reflect(1, reverse=False)	Flips a grid vertically and appends to below the original grid
RandomTranslateXY()	Shifts a grid randomly in the horizontal and vertical directions. The maximum shift size is 4
Transpose()	Reflects a grid on diagonal
IncreaseResolution(2)	Upscales the grid by interleaving elements in both horizontal and vertical directions
IncreaseHeight(2)	Upscales the grid by interleaving elements in vertical direction
IncreaseWidth(2)	Upscales the grid by interleaving elements in horizontal direction
Chain([Rotate(90),IncreaseResolution(2)])	Sequential application of Rotate(90) and IncreaseResolution(2)
Chain([Rotate(270),IncreaseResolution(2)])	Sequential application of Rotate(270) and IncreaseResolution(2)
Chain([Rotate(180),IncreaseResolution(2)])	Sequential application of Rotate(180) and IncreaseResolution(2)
Chain([Flip(0),IncreaseResolution(2)])	Sequential application of Rotate(180) and IncreaseResolution(2)
Chain([Flip(1),IncreaseResolution(2)])	Sequential application of Rotate(180) and IncreaseResolution(2)
Chain([Transpose(),IncreaseResolution(2)])	Sequential application of Rotate(180) and IncreaseResolution(2)

表 6. ARC TTT 超参数。我们发现学习率 $5e-5$ 对 1B 和 3B 模型最佳, $1e-4$ 对 8B 模型最佳。

Hyperparameter	Search Space
r LoRA rank	128
α LoRA alpha	16
learning rate	[$5e-5$, $1e-4$]
epochs	2
batch size	[1, 2]
optimizer	AdamW (Loshchilov & Hutter, 2018)

D. LM 数据生成

我们在附录 B 中描述了使用 LM 的三种方法，通过少样本提示 GPT-4 和 GPT-4o 模型（OpenAI，2024；Hurst 等人，2024）生成了 6426 个任务生成器。

D.1. 获取任务描述

该过程如图 15 所示。我们最初以图 11 所示的分层风格描述了 10 个训练任务。随后，对于其他训练任务，我们从 LARC（Acquaviva 等人，2022 年）项目中获取了质量较低的众包工作者标注。通过利用我们的高质量种子标注及其 LARC 版本，我们使用 10 次提示和语言模型为其他训练任务生成高质量标注。

你是一个能够从示例中归纳任务描述的智能代理。对于类别，请*不要*使用诸如“变换”、“模式识别”之类的通用术语。——任务：{ 字符串化的任务输入和输出 } LARC 描述：{ 来自 LARC 数据集的任务-1 的描述 } 良好描述：{ 分层描述 } ———— [truncated] ———— 任务：{ 任务-K 的字符串化任务输入和输出 } LARC 描述：{ 来自 LARC 数据集的任务-K 的描述 } 良好描述：{ 分层描述 } —— 任务：{ 查询任务的字符串化任务输入和输出 } LARC 描述：{ 来自 LARC 数据集的查询任务的描述 }

D.2. 少样本提示细节

在所有数据生成过程中，我们采用以下简单的提示模板，并配合 k 样本提示，其中数字由从种子集采样的示例填充。在简单的少样本生成中，我们排除示例。我们使用 GPT-4 和 GPT-4o 来生成新的脚本。

你是一个在二维颜色网格上的问题生成器。以下是一些此类变换的示例，请遵循以下格式：——— 示例：{ 生成器函数-1 的描述 } 脚本：{ 生成器函数-1 } ———— [已截断] ———— 示例：{ 生成器函数-K 的描述 } 脚本：{ 生成器函数-K }

请生成更多内容，并确保它们各不相同：

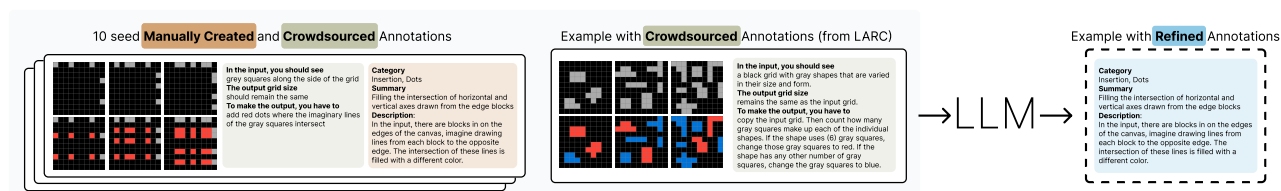


图 15. 生成高质量种子描述：本文采用少样本提示方法，为给定任务生成描述，使用 10 个手工创建的种子描述以及来自 Acquaviva 等人（2022）的众包工作者标注作为少样本示例。对于给定的新任务，本文同样向语言模型提供示例和众包工作者标注（仅训练任务可用）。

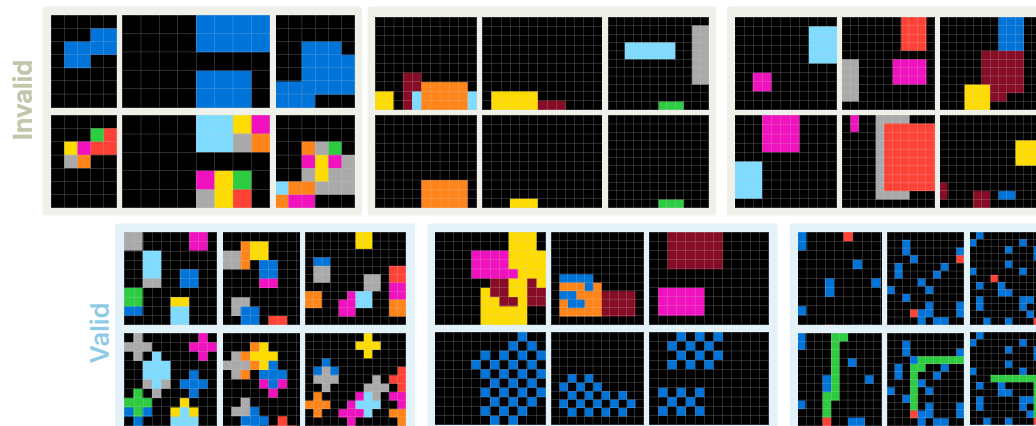


图 16. 由语言模型数据增强程序生成的任务示例：我们展示了三个可以推断出简单变换的合理任务（有效），以及三个无法推断出简单变换的任务（无效）。

E. 增强推理流程

E.1. 增强推理

近期研究表明，扩展测试时计算可以显著提升语言模型的性能。最常用的技术之一是通过采样多个响应，然后使用排序器选择最佳响应。然而，尽管采样在具有多个可能解（如代码中的程序）或通往最终答案的多条路径（如数学）的领域中非常有效，但在直接生成答案时可能有害，因为无法在确保样本内部一致性的同时直接强制样本间的多样性。作为推理时扩展的替代方案，我们采用一种增强推理策略，利用几何变换生成多个预测候选，并结合贪心解码方案。

对于具有训练示例 $(x_k, y_k)_{k=1}^K$ 和测试输入 x_{test} 的给定任务，我们使用可逆几何变换生成任务的等价变换版本，如图 5 所示。设 $\mathcal{T}$ 为某个可逆几何变换集合（例如旋转和反射）。对于每个变换 $t \in \mathcal{T}$ ，我们将 t 应用于所有训练演示和测试输入，并使用这些变换后的输入运行我们的模型。然后，我们应用逆变换以获得该变换下的最终预测。

$$\tilde{y} \sim \text{LM}(t(\mathbf{d}_{\text{input}})) := [t(x_1), t(y_1), \dots, t(x_{\text{test}})] \quad (6)$$

$$y_t = t^{-1}(\tilde{y}) \quad (7)$$

我们进一步通过排列训练示例的顺序来增强预测。对于每个变换 g ，我们采样 $n = 2$ 个不同的演示序列排列，从而每个任务产生 $n \cdot |\mathcal{T}|$ 个总预测。这是为了减轻模型在处理演示序列时的任何偏差。（Bober-Irizar & Banerjee, 2024）也发现转置和旋转有助于产生额外的预测候选。

E.2. 集成预测（投票策略）

我们采用分层投票策略从候选集 $\{y\}_{i=1}^{n \cdot |\mathcal{T}|}$ 中确定最终预测。

该方法

包含两轮投票，以逐步缩小最佳候选范围：首先在每个变换内部选取出现频率最高的预测，随后对所有变换专属候选进行整体投票，以确定出现频率最高的前两个预测。各阶段细节如下：

1. 变换内投票：按对应变换 t 对预测进行分组，并在每组内选取出现频率最高的前三个预测。若某组内不同预测少于三个，则通过以下方式补充候选：

- **基于行的多数投票：**对于预测输出网格中的每一行，取该变换组内所有预测中该行出现频率最高的行值。
- **基于列的多数投票：**类似地，对于预测输出网格中的每一列，取该变换组内所有预测中该列出现频率最高的列值。

2. 全局投票：利用第（1）步得到的变换专属候选，进行整体投票，以选出提交用出现频率最高的前两个预测。若出现平局，则优先选择恒等变换对应的预测。

F. BIG-Bench Hard 细节

F.1. 进一步实验细节

我们为 BIG-Bench Hard 编写了自己的评估函数，该函数可在我们的代码库中找到。我们发现现有的评估框架由于答案提取解析和答案格式提示不足，未能正确衡量零样本性能。我们还希望更灵活地将每个单独任务的数据集划分为演示示例和评估集。对于所有结果，我们使用以下 5 个随机种子对 10 个少样本示例的不同选择进行平均：42, 43, 44, 45, 46。完整的 TTT 和推理过程在 NVIDIA A100 GPU 上大约需要 15 分钟。

图 8 中每种方法在 5 个种子上的均值标准误差见表 7。

表 7. 图 8 中每种方法的均值标准误差。方法 均值标准误差

Method	Standard Error of the Mean
Zero-Shot	0.01
ICL	0.19
TTT	0.20
No Example Permutation	0.32
E2E	0.66
Shared TTT	0.72
No Demo Loss	0.69
Loss on Inputs and Outputs	0.35

我们搜索以下超参数：

表 8. BBH TTT 微调超参数 超参数 搜索空间

Hyperparameter	Search Space
learning rate	[1e-5, 5e-5, 1e-4 , 3e-4]
r LoRA rank	[64 , 128]
α LoRA alpha	[16, 32, 64 , 128]
epochs	1
batch size	5
training steps	[20, 40 , 60]
optimizer	AdamW
scheduler	Cosine LR Schedule

我们同样使用 torchtune (torchtune Maintainers & Contributors, 2024) 库进行测试时训练，并使用 vLLM (Kwon et al., 2023) 库进行推理。

F.2. 任务特定结果

所有任务在所有方法和消融实验上的完整结果如图 17 所示。

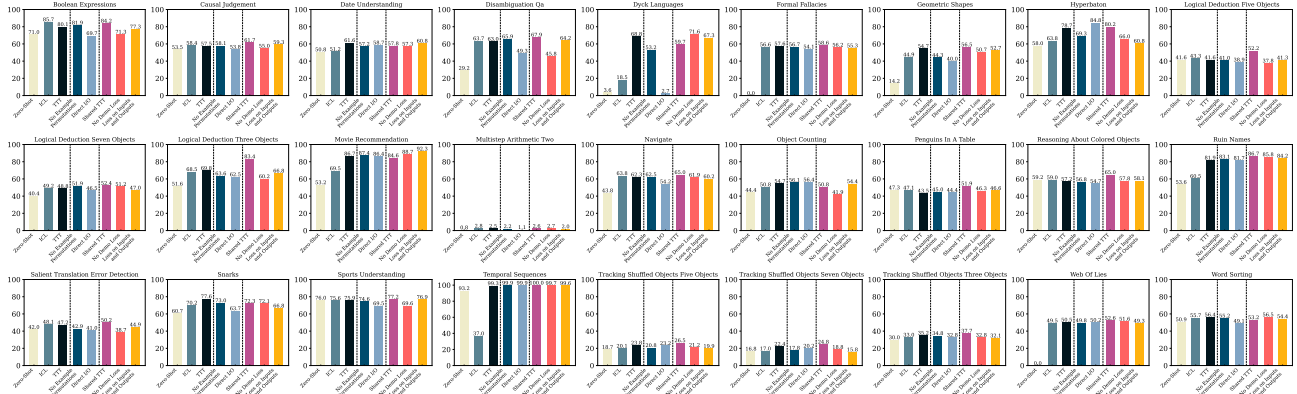


图 17. 每个 BIG-Bench Hard 任务的任务特定 10 样本结果，在 5 个随机种子上取平均。

范围	$x = \text{TTT 准确率} - \text{ICL 准确率}$	任务 (数量)
$x \leq -5$		1 task total: <i>Boolean Expressions</i>
$-5 < x \leq -2$		1 task total: <i>Penguins In A Table</i>
$-2 < x < 2$		共 13 个任务：因果判断、歧义问答、形式谬误、五物体逻辑演绎、七物体逻辑演绎、三物体逻辑演绎、两步多步算术、导航、彩色物体推理、显著翻译错误检测、体育理解、谎言网络、单词排序 共 3 个任务：物体计数、五物体乱序追踪、三物体乱序追踪 共 9 个任务：日期理解、戴克语言、几何形状、倒置法、电影推荐、名称毁坏、蛇鲨、时间序列、七物体乱序追踪
$2 \leq x < 5$		
$x \geq 5$		

表 9. 按 $x = \text{TTT 准确率} - \text{ICL 准确率}$ 差异分类的任务。